

Homework #1

Problem #1:

Step 1: Define the decision variables

Let $x_i = \text{number of shares of stock } i \text{ to sell}$ for $i=1, \dots, 10$.

Since the investor owns 100 shares of each stock, the decision variables must satisfy:

$$0 \leq x_i \leq 100$$

Step 2: Define the objective function

The goal is to maximize the expected value of the remaining portfolio one year from now. If the expected price of stock i (P_i) in one year, then the number of shares remaining is:

$$\max \sum_{i=1}^{10} (100 - x_i)P_i$$

Since, $\sum 100 * P_i$ is constant, this is equivalent to minimizing the expected future value lost from selling shares:

$$\min \sum_{i=1}^{10} x_i P_i$$

This second version is useful because Matlab's *linprog* solves minimization problems by default.

Step3: Define the cash constraint

The investor must raise at least \$30,000 after transaction costs and taxes. For each share sold, the investor receives the current price:

$$P_i^{\text{current}}$$

However, there is a 1% transaction cost:

$$0.01 * P_i^{\text{current}}$$

There is also a 30% tax on the capital gain:

$$0.30 * (P_i^{\text{current}} - P_i^{\text{purchase}})$$

So, the net cash received per share sold is:

$$P_i^{current} - 0.01 * P_i^{current} - 0.30 * (P_i^{current} - P_i^{purchase})$$

The total net cash raised must satisfy:

$$\sum_{i=1}^{10} x_i * (P_i^{current} - 0.01 * P_i^{current} - 0.30 * (P_i^{current} - P_i^{purchase})) \geq 30000$$

In short, the problem is:

$$\min \sum_{i=1}^{10} x_i P_i$$

Subject to:

$$\sum_{i=1}^{10} x_i * (P_i^{current} - 0.01 * P_i^{current} - 0.30 * (P_i^{current} - P_i^{purchase})) \geq 30000$$

$$0 \leq x_i \leq 100, i = 1, \dots, 10$$

Results:

The results indicate that the investor should fully liquidate stocks 3, 4, 8, 9 and 10 (sell all 100 shares of each. In addition, approximately 63.75 shares of stock 6 should be sold. All remaining stock: 1, 2, 5 and 7 should be held entirely.

This strategy generates a total net cash inflow of exactly \$30,000, thereby satisfying the cash constraint with equality. The expected value of the remaining portfolio after one year is \$20,893.71.

Problem #2:

The problem cannot be formulated as a linear programming problem. A linear program must have a linear objective function and linear constraints. However, the objective function contains the nonlinear term:

$$x_2^2$$

Therefore, the objective function is not linear.

In addition, the constraint:

$$x_2^2 x_3 = 0$$

Is also nonlinear because it contains a squared variable and a product of variables. Since, $x_2 \geq 0$ and $x_3 \geq 0$, this constraint implies that either:

$$x_2 = 0$$

Or

$$x_3 = 0$$

This creates a nonconvex feasible region, which cannot be represented as a standard linear programming feasible region. Therefore, the given optimization problem is not a linear programming problem and cannot be transformed into a linear program in standard form.

Problem #3:

a)

Let $x_{ij} = 1$, if worker i is assigned to project j and 0 otherwise.

Where $i=1, \dots, 4$ and $j=1, \dots, 4$.

Since each worker must be assigned to exactly one project:

$$\sum_{j=1}^4 x_{ij} = 1, i = 1, \dots, 4$$

Since each project must be assigned exactly one worker:

$$\sum_{i=1}^4 x_{ij} = 1, j = 1, \dots, 4$$

The objective is to minimize the total cost. Since each worker costs \$20 per hour, the cost of assigning worker i to project j is:

$$20t_{ij}$$

Where t_{ij} is the number of hours required.

Therefore, the linear programming model is:

$$\min \sum_{i=1}^4 \sum_{j=1}^4 20t_{ij}x_{ij}$$

Subject to

$$\begin{aligned} \sum_{j=1}^4 x_{ij} &= 1, i = 1, \dots, 4 \\ \sum_{i=1}^4 x_{ij} &= 1, j = 1, \dots, 4 \\ 0 &\leq x_{ij} \leq 1 \end{aligned}$$

b)

Student : Jerome Charbonneau

The optimal solution is:

$$x_{12} = 1, x_{21} = 1, x_{33} = 1, x_{44} = 1$$

This means:

Worker 1 is assigned to Project 2, Worker 2 is assigned to Project 1, Worker 3 is assigned to Project 3 and Worker 4 is assigned to Project 4.

The total number of hours is:

$$3 + 5 + 7 + 8 = 23 \text{ hours}$$

Since the cost is \$20 per hour, the minimum total cost is: $23 * 20 = 460$

Therefore, the optimal objective function value is: 460.

Problem #4:

This problem is an inventory management linear programming problem. The goal is to maximize the trader's total profit over 12 months.

Step 1: Decision variables

Let:

$$\begin{aligned} b_t &= \text{number of units bought in month } t \\ s_t &= \text{number of units sold in month } t \\ I_t &= \text{inventory at the end of month } t \\ &\text{for } t = 1, 2, \dots, 12. \end{aligned}$$

The initial inventory is:

$$I_0 = 2000$$

Step 2: Objective function

The trader earns revenue from selling units, pays for buying units and pays holding costs on inventory.

The objective is:

$$\max \sum_{t=1}^{12} (r_t s_t - p_t b_t - h_t I_t)$$

Where:

$$\begin{aligned}r_t &= \text{selling price in month } t \\p_t &= \text{purchase price in month } t \\h_t &= \text{holding cost in month } t\end{aligned}$$

Step 3: Inventory balance constraints

For each month:

$$I_t = I_{t-1} + b_t - s_t$$

For

$$t = 1, \dots, 12$$

This ensures that inventory is updated correctly each month.

Step 4: Capacity constraints

The warehouse can hold at most 10,000 units:

$$I_t \leq 10000$$

For:

$$t = 1, \dots, 12$$

Step 5: Final inventory constraint

The trader must end the year with no inventory:

$$I_{12} = 0$$

Step 6: Non-negativity constraints

The decision variables cannot be negative:

$$b_t \geq 0, s_t \geq 0, I_t \geq 0$$

For:

$$t = 1, \dots, 12$$

The final linear program maximizes total profit subject to inventory balance, warehouse capacity, final inventory and non-negativity constraints. All terms are linear, so this is a valid linear programming formulation.

Problem #5:

Let: x_1, x_2, x_3 be the amount invested in Bonds 1, 2 and 3.

Let: e_1, e_2 be the excess cash generated at the end of years 1 and 2, respectively which is reinvested at 2% for one period.

The objective is to minimize the total cost of the bond portfolio:

$$\min 102 * x_1 + 99 * x_2 + 98 * x_3$$

Subject to:

$$\begin{aligned} 105 * x_1 + 3.5 * x_2 + 3.5 * x_3 - e_1 &= 12000 \\ 103.5 * x_2 + 3.5 * x_3 + 1.02 * e_1 - e_2 &= 18000 \\ 103.5 * x_3 + 1.02 * e_2 &\geq 20000 \\ x_1, x_2, x_3, e_1, e_2 &\geq 0 \end{aligned}$$

The optimal solution is:

$$\begin{aligned} x_1 &= 102.2652 \\ x_2 &= 167.3785 \\ x_3 &= 193.2367 \\ e_1 &= 0, e_2 = 0 \end{aligned}$$

The minimum portfolio cost is: 45,938.72. Allowing excess cash generation and reinvestment is advantageous because it gives the investor more flexibility. If a bond portfolio generates more cash than required in an earlier period, the excess can be invested and used to help satisfy future liabilities. This can potentially reduce the total cost of the portfolio. In this specific case, however the optimal solution does not generate excess cash, so the reinvestment variables are equal to zero.

Problem #6:

Let:

$$\begin{aligned} x_1 &= \text{flights from Calgary to Winnipeg} \\ x_2 &= \text{flights from Winnipeg to Montreal} \\ x_3 &= \text{flights from Winnipeg to Hamilton} \\ x_4 &= \text{flights from Winnipeg to Toronto} \\ x_5 &= \text{flights from Winnipeg to Quebec city} \\ x_6 &= \text{flights from Hamilton to Quebec city} \\ x_7 &= \text{flights from Toronto to Quebec city} \end{aligned}$$

Student : Jerome Charbonneau

Linear programming formulation:

$$\max x_5 + x_6 + x_7$$

Subject to flow conservation:

$$x_1 - x_2 - x_3 - x_4 = 0$$

$$x_2 - x_5 = 0$$

$$x_3 - x_6 = 0$$

$$x_4 - x_7 = 0$$

Capacity constraints:

$$0 \leq x_1 \leq 5, 0 \leq x_2 \leq 4, 0 \leq x_3 \leq 5, 0 \leq x_4 \leq 2, 0 \leq x_5 \leq 2,$$

$$0 \leq x_6 \leq 1, 0 \leq x_7 \leq 3$$

One optimal solution is:

$$x_1 = 5, x_2 = 2, x_3 = 1, x_4 = 2, x_5 = 2, x_6 = 1, x_7 = 2$$

Therefore, the maximum number of daily flights from Calgary to Quebec city is 5.

Appendix (Question 1):

```
Homework1.m x +
IATLAB Drive/Homework1.m
1 % Homework 1 - Problem 1
2 % Linear Programming Formulation
3
4 % =====
5 % Data
6 % =====
7
8 shares_owned = 100;
9
10 purchase_price = [20; 25; 30; 35; 40; 45; 50; 55; 60; 65];
11 current_price = [30; 34; 43; 47; 49; 53; 60; 62; 64; 66];
12 future_price = [36; 39; 42; 45; 51; 55; 63; 64; 66; 70];
13
14 tax_rate = 0.30;
15 transaction_cost = 0.01;
16
17 n = length(purchase_price);
18
19 % =====
20 % Net cash received per share sold
21 % =====
22
23 net_cash = current_price ...
24     - transaction_cost .* current_price ...
25     - tax_rate .* (current_price - purchase_price);
26
27 % =====
28 % Objective function
29 % =====
30 % MATLAB linprog minimizes.
31 % Maximizing the future value of remaining shares is equivalent to
32 % minimizing the future value lost from selling shares.
33
34 f = future_price;
```

Student : Jerome Charbonneau

\\B Drive\Homework1.m

```
% Minimizing the future value loss from selling shares.

f = future_price;

% =====
% Constraint
% =====
% Need at least $30,000 after taxes and transaction costs:
%
% net_cash' * x >= 30000
%
% linprog requires A*x <= b, so multiply by -1:
%
% -net_cash' * x <= -30000

A = -net_cash';
b = -30000;

% =====
% Bounds
% =====
% We cannot sell fewer than 0 shares or more than 100 shares.

lb = zeros(n,1);
ub = shares_owned * ones(n,1);

% =====
% Solve linear program
% =====

options = optimoptions('linprog','Display','iter');

[x, fval, exitflag, output] = linprog(f, A, b, [], [], lb, ub, options);

% =====
% Compute final results
% =====

cash_raised = net_cash' * x;

future_value_remaining = future_price' * (shares_owned * ones(n,1) - x);

shares_remaining = shares_owned * ones(n,1) - x;

% =====
% Display results
% =====

disp('Optimal number of shares to sell:');
disp(x);

disp('Shares remaining after selling:');
disp(shares_remaining);

disp('Total net cash raised:');
disp(cash_raised);

disp('Expected future value of remaining portfolio:');
disp(future_value_remaining);

disp('Exit flag:');
disp(exitflag);
```

Student : Jerome Charbonneau

```
Command Window

New to MATLAB? See resources for Getting Started

Optimal solution found.

Optimal number of shares to sell:
0
0
100.0000
100.0000
0
63.7507
0
100.0000
100.0000
100.0000

Shares remaining after selling:
100.0000
100.0000
0
0
100.0000
36.2493
100.0000
0
0
0
```

```
Exit flag:
1
```

Stock	purchase_price	current_price	future_price	net_cash	x	shares_remaining
1	20	30	36	26.7	0	100
2	25	34	39	30.96	0	100
3	30	43	42	38.67	100	0
4	35	47	45	42.93	100	0
5	40	49	51	45.81	0	100
6	45	53	55	50.07	63.751	36.249
7	50	60	63	56.4	0	100
8	55	62	64	59.28	100	0
9	60	64	66	62.16	100	0
10	65	66	70	65.04	100	0

```
>>
```

Appendix (Question 3):

```
% Homework 1 – Problem 3
% Exercise 1.8 – Assignment Problem

clear;
clc;

% Time required in hours
time = [7 3 6 10;
        5 4 9 9;
        6 4 7 10;
        5 5 6 8];

% Cost per hour
cost_per_hour = 20;

% Cost matrix
cost = cost_per_hour * time;

% Convert cost matrix into vector for linprog
f = cost(:);

% Number of workers and projects
n = 4;

% Equality constraints
% Each worker assigned to exactly one project
% Each project assigned to exactly one worker

Aeq = [];

% Worker constraints
for i = 1:n
    row = zeros(n,n);
    row(i,:) = 1;
    Aeq = [Aeq; row(:)'];
    beq = [beq; 1];
end

% Project constraints
for j = 1:n
    row = zeros(n,n);
    row(:,j) = 1;
    Aeq = [Aeq; row(:)'];
    beq = [beq; 1];
end

% Bounds
lb = zeros(n*n,1);
ub = ones(n*n,1);

% Solve using linprog
[x, fval] = linprog(f, [], [], Aeq, beq, lb, ub);

% Reshape solution into 4x4 assignment matrix
X = reshape(x, n, n);

% Display results
disp('Optimal assignment matrix:');
disp(X);

disp('Minimum total cost:');
disp(fval);
```

Appendix (Question 5):

```
% Homework 1 - Problem 5
% Exercise 1.14 - Assignment Problem

f = [102 99 98 0 0];

A = [0 0 -103.5 0 -1.02];
b = [-20000];

Aeq = [105 3.5 3.5 -1 0;
       0 103.5 3.5 1.02 -1];

beq = [12000; 18000];

lb = zeros(5,1);

[x, fval] = linprog(f, A, b, Aeq, beq, lb);

x
fval
```

Appendix (Question 6):

```
% Homework 1 - Problem 6
% Exercise 1.18 - Assignment Problem

f = [0 0 0 0 -1 -1 -1];

Aeq = [1 -1 -1 -1 0 0 0;
       0 1 0 0 -1 0 0;
       0 0 1 0 0 -1 0;
       0 0 0 1 0 0 -1];

beq = [0; 0; 0; 0];

lb = zeros(7,1);

ub = [5; 4; 5; 2; 2; 1; 3];

[x, fval] = linprog(f, [], [], Aeq, beq, lb, ub);

maxFlights = -fval;

x
maxFlights
```